

Intervenants: Felipe FIGUEREDO-ROCHA (felipe.figuereredo-rocha@u-pec.fr)

Méthodes de résolution numérique de problèmes d'analyse

TP1 Introduction pytorch, régression et classification

1 Contexte et objectifs

L'apprentissage automatique (*Machine Learning*, ML) est de plus en plus utilisé en mécanique numérique pour construire des modèles de substitution, identifier des lois constitutives ou accélérer des simulations numériques.

L'objectif de ce TP est de découvrir les principales étapes d'une démarche d'apprentissage automatique à travers deux problèmes directement liés à la mécanique :

1. une **régression** permettant d'approximer une relation contrainte-déformation à partir d'une loi hyperélastique de type Neo-Hookean ;
2. une **classification** permettant de déterminer si un état de contrainte satisfait ou non un critère de rupture/plastification, basé sur la contrainte von Mises.

Dans les deux cas, un modèle mécanique de référence est utilisé pour générer les données synthétiques.

2 Environnement Python et Pytorch

Le TP utilise principalement les libraries `numpy`, `matplotlib`, `pytorch` et ces composantes. Veuillez consulter la documentation en fonction de votre version : <https://docs.pytorch.org/docs/2.14/index.html> (v2.14). Pour obtenir des résultats reproductibles, veuillez définir une *seed* pour le générateur de nombres aléatoires interne de PyTorch :

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
print(torch.__version__)
torch.manual_seed(45) # 45 pourra etre remplace par d'autre entier positif
```

3 Étude 1 – Régression d'une loi hyperélastique

On souhaite construire un réseau de neurones capable de prédire la contrainte à partir de la déformation :

$$\mathbf{F} \longrightarrow \text{réseau de neurones} \longrightarrow \hat{\boldsymbol{\sigma}}$$

Le réseau ne connaît pas explicitement la loi mécanique. Il dispose uniquement d'un ensemble d'exemples :

$$\left\{ (\mathbf{F}^{(i)}, \boldsymbol{\sigma}^{(i)}) \right\}_{i=1}^N.$$

3.1 Loi de comportement Neo-Hookean

On considère l'énergie de déformation compressible :

$$\Psi(\mathbf{F}) = \frac{\mu}{2}(I_1 - 3) - \mu \ln J + \frac{\lambda_L}{2}(\ln J)^2,$$

avec

$$\mathbf{C} = \mathbf{F}^T \mathbf{F}, \quad J = \det \mathbf{F} > 0, \quad I_1 = \text{tr}(\mathbf{C}).$$

Le premier tenseur de Piola–Kirchhoff est défini par

$$\mathbf{P} = \frac{\partial \Psi}{\partial \mathbf{F}} = \mu \mathbf{F} + (\lambda_L \ln J - \mu) \mathbf{F}^{-T}$$

Enfin, le tenseur de Cauchy s’obtient par la transformation

$$\boldsymbol{\sigma} = \frac{1}{J} \mathbf{P} \mathbf{F}^T = \frac{\mu}{J} (\mathbf{b} - \mathbf{I}) + \frac{\lambda_L \ln J}{J} \mathbf{I},$$

où le tenseur de Cauchy–Green gauche est donné par

$$\mathbf{b} = \mathbf{F} \mathbf{F}^T.$$

3.2 Tutoriel 1 : Introduction au Pytorch pour la régression

Dans cette partie TP introductif, nous considérons une traction uniaxiale et utilisons la relation simplifiée, avec $\mu = 10$ MPa.

$$\sigma(\lambda) = \mu \left(\lambda^2 - \frac{1}{\lambda} \right).$$

3.2.1 Génération des données

Créer $N = 500$ valeurs de l’allongement dans

$$1 \leq \lambda \leq 2.$$

```
N = 500
mu = 10.0
lam = np.linspace(1.0, 2.0, N)
sigma = mu * (lam**2 - 1.0 / lam)
plt.figure()
plt.plot(lam, sigma)
plt.xlabel(r"$\lambda$")
plt.ylabel(r"$\sigma$ [MPa]")
plt.grid()
plt.show()
```

Pour représenter des données expérimentales, ajouter un bruit gaussien :

$$\sigma_{\text{data}} = \sigma + \eta, \quad \eta \sim \mathcal{N}(0, s^2).$$

Par exemple :

```
rng = np.random.default_rng(42)
noise = rng.normal(loc=0.0, scale=0.5, size=N)
sigma_data = sigma + noise

plt.figure()
plt.plot(lam, sigma, label="Neo-Hookean")
plt.scatter(lam, sigma_data,
            s=10, label="Donnees bruitees")
plt.xlabel(r"$\lambda$")
plt.ylabel(r"$\sigma$ [MPa]")
plt.legend()
plt.grid()
plt.show()
```

3.2.2 Conversion en tenseurs PyTorch

PyTorch travaille avec des tenseurs.

```
X = torch.tensor(lam, dtype=torch.float32).reshape(-1, 1)
y = torch.tensor(sigma_data, dtype=torch.float32).reshape(-1, 1)
print(X.shape)
print(y.shape)
```

On obtient ici :

$$\mathbf{X} \in \mathbb{R}^{N \times 1}, \quad \mathbf{y} \in \mathbb{R}^{N \times 1}.$$

À vous de réfléchir

Quelle est la différence entre un tableau NumPy et un tenseur PyTorch ?
 Pourquoi PyTorch est-il particulièrement adapté à l'entraînement de réseaux de neurones ?
 Pourquoi utilise-t-on la forme $(N, 1)$ plutôt qu'un vecteur de taille N ?

3.2.3 Séparation apprentissage / test

On réserve une partie des données au test.

```
indices = torch.randperm(N)
n_train = int(0.8 * N)
train_idx = indices[:n_train]
test_idx = indices[n_train:]
X_train = X[train_idx]
y_train = y[train_idx]
X_test = X[test_idx]
y_test = y[test_idx]
```

Point d'attention

Le jeu de test ne doit pas être utilisé pendant l'optimisation des paramètres du réseau.

3.2.4 Premier réseau de neurones

Un réseau de neurones peut être représenté sous la forme

$$\lambda \rightarrow \text{couche cachée} \rightarrow \text{couche cachée} \rightarrow \hat{\sigma}.$$

Nous définissons le modèle avec PyTorch :

```
class RegressionNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(1, 32),
            nn.Tanh(),
            nn.Linear(32, 32),
            nn.Tanh(),
            nn.Linear(32, 1)
        )

    def forward(self, x):
        return self.network(x)

model = RegressionNetwork()
print(model)
```

3.3 Fonction de coût/Optimisation

Pour la régression, on utilise l'erreur quadratique moyenne :

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{\sigma}_i - \sigma_i)^2.$$

Les paramètres du réseau sont ajustés avec l'algorithme Adam.

```
loss_function = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
```

La boucle suivante contient les étapes fondamentales de l'apprentissage :

prédiction → fonction de coût → gradient → mise à jour

```
n_epochs = 3000
loss_history = []
for epoch in range(n_epochs):
    optimizer.zero_grad()
    y_pred = model(X_train)

    loss = loss_function(y_pred, y_train)
    loss.backward()

    optimizer.step()

    loss_history.append(loss.item())

    if epoch % 200 == 0:
        print(epoch, loss.item())
```

À vous de réfléchir

À quoi sert les commande `loss.backward()` et `optimizer.step()` ? À quoi sert la commande `optimizer.zero_grad()`. Que se passerait-il si elle était supprimée ?

3.4 Évaluation du modèle

Tracer l'évolution de la fonction de coût :

```
plt.figure()
plt.semilogy(loss_history)
plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.grid()
plt.show()
```

À vous de réfléchir

Que peut-on observer sur la convergence ? Comment reconnaître une situation dans laquelle l'entraînement ne progresse plus ?

Mettre le réseau en mode évaluation et calculer la MSE :

```
model.eval()
with torch.no_grad():
    y_pred_test = model(X_test)

mse = torch.mean((y_pred_test - y_test)**2)
rmse = torch.sqrt(mse)
```

```
mae = torch.mean(torch.abs(y_pred_test - y_test))

print("MSE_λ=", mse.item())
print("RMSE_λ=", rmse.item())
print("MAE_λ=", mae.item())
```

Comparaison avec la loi mécanique :

```
lam_plot = torch.linspace(1.0, 2.0, 300).reshape(-1, 1)

with torch.no_grad():
    sigma_ml = model(lam_plot)

sigma_exact = mu * (lam_plot**2 - 1.0 / lam_plot)

plt.figure()
plt.scatter(X_train.numpy(), y_train.numpy(), s=10, label="Train")
plt.scatter(X_test.numpy(), y_test.numpy(), s=10, label="Test")
plt.plot(lam_plot.numpy(), sigma_exact.numpy(), label="Neo-Hookean")
plt.plot(lam_plot.numpy(), sigma_ml.numpy(), "--", label="PyTorch")

plt.xlabel(r"$\lambda$")
plt.ylabel(r"$\sigma_\lambda$ [MPa]")
plt.legend()
plt.grid()
plt.show()
```

À vous de réfléchir

Pourquoi l'extrapolation est-elle particulièrement problématique pour un modèle constitutif destiné à être utilisé dans un calcul éléments finis ?

3.5 Extension : Apprendre une loi constitutive

L'objectif de départ est d'apprendre

$$\mathbf{F} \longrightarrow \boldsymbol{\sigma}$$

Une représentation directe consiste à utiliser les neuf composantes de $\mathbf{F}$:

$$\mathbf{x} = (F_{11}, F_{12}, \dots, F_{33})$$

et à prédire les composantes indépendantes de la contrainte :

$$\mathbf{y} = (\sigma_{11}, \sigma_{22}, \sigma_{33}, \sigma_{12}, \sigma_{13}, \sigma_{23}).$$

À vous de jouer

Proposer une architecture PyTorch pour résoudre ce problème. Discuter :

1. la génération de données ;
2. le nombre de couches cachées ;
3. la fonction d'activation ;
4. la fonction de coût ;
5. la normalisation des données ;
6. le nombre de données nécessaires ;
7. l'extrapolation et validation du modèle.

4 Étude 2 – Classification d'un état de rupture

On cherche maintenant à apprendre une fonction de classification :

$$(\sigma_{xx}, \sigma_{yy}, \tau_{xy}) \longrightarrow \{\text{admissible, rupture}\}$$

Les étiquettes sont générées à partir d'un critère mécanique de von Mises. Pour un état de contrainte plane

$$\boldsymbol{\sigma} = \begin{pmatrix} \sigma_{xx} & \tau_{xy} & 0 \\ \tau_{xy} & \sigma_{yy} & 0 \\ 0 & 0 & 0 \end{pmatrix},$$

la contrainte équivalente de von Mises vaut

$$\sigma_{\text{VM}} = \sqrt{\sigma_{xx}^2 - \sigma_{xx}\sigma_{yy} + \sigma_{yy}^2 + 3\tau_{xy}^2}.$$

4.1 Tutoriel classification : Critère de von Mises

4.1.1 Génération des données

On considère un seuil

$$\sigma_y = 100 \text{ MPa.}$$

On définit alors

$$y = \begin{cases} 0, & \sigma_{\text{VM}} < \sigma_y, \\ 1, & \sigma_{\text{VM}} \geq \sigma_y. \end{cases}$$

```
N = 5000
s_xx, s_yy, t_xy = np.random.default_rng(42).uniform(-150, 150, (3, N))
sigma_vm = np.sqrt(s_xx**2 - s_xx * s_yy + s_yy**2 + 3 * t_xy**2)

X = torch.tensor(np.column_stack([s_xx, s_yy, t_xy]), dtype=torch.float32)
y = torch.tensor((sigma_vm >= 100.0).astype(np.float32), dtype=torch.float32).reshape(-1, 1)
```

4.1.2 Split et Normalisation

Les trois composantes de contrainte n'ont pas nécessairement la même échelle. On peut normaliser les données à partir de la moyenne et de l'écart-type du jeu d'apprentissage :

$$\tilde{x} = \frac{x - \mu_x}{s_x}.$$

```
indices = torch.randperm(N)
n_train = int(0.8 * N)
train_idx, test_idx = indices[:n_train], indices[n_train:]

mean, std = X[train_idx].mean(0), X[train_idx].std(0)
X_train, X_test = (X[train_idx] - mean) / std, (X[test_idx] - mean) / std
y_train, y_test = y[train_idx], y[test_idx]
```

4.1.3 Réseau de classification et entraînement

On construit un réseau :

$$\mathbb{R}^3 \rightarrow \mathbb{R}^{32} \rightarrow \mathbb{R}^{32} \rightarrow \mathbb{R}.$$

```
model = nn.Sequential(nn.Linear(3, 32), nn.Tanh(), nn.Linear(32, 32), nn.Tanh(), nn.Linear(32, 1))
```

Pour la fonction coût, on utilise BCEWithLogitsLoss, qui combine numériquement la fonction sigmoïde et l'entropie croisée binaire. L'optimiser c'est le Adam.

```
criterion, optimizer = nn.BCEWithLogitsLoss(), optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(2000):
    optimizer.zero_grad()
    loss = criterion(model(X_train), y_train)
    loss.backward()
    optimizer.step()
    if epoch % 200 == 0: print(f"Epoch_{epoch}, Loss={loss.item():.4f}")
```

Point d'attention

La dernière couche produit un *logit* $z \in \mathbb{R}$. La probabilité de rupture est donné par la sigmoïde

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + \exp(-z)}.$$

4.1.4 Evaluation du modèle

Le réseau produit un logit. On applique ensuite une sigmoïde avec la règle de décision

$$\hat{y} = \begin{cases} 1 & P(y = 1|\mathbf{x}) \geq 0.5, \\ 0 & P(y = 1|\mathbf{x}) < 0.5. \end{cases}$$

et l'*accuracy* donné par

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

```
model.eval()
with torch.no_grad():
    preds = (torch.sigmoid(model(X_test)) >= 0.5).float()

TP = ((preds == 1) & (y_test == 1)).sum().item()
TN = ((preds == 0) & (y_test == 0)).sum().item()
FP = ((preds == 1) & (y_test == 0)).sum().item()
FN = ((preds == 0) & (y_test == 1)).sum().item()
print(f"TP={TP}, TN={TN}, FP={FP}, FN={FN}")
print(f"Accuracy={((preds==y_test).float().mean().item():.4f}")
```

À vous de réfléchir

Dans une application mécanique, quelle erreur peut être la plus problématique : *FP* ou *FN* ?

4.1.5 Visualisation de la frontière de classification

Pour simplifier la visualisation, on fixe $\tau_{xy} = 0$ et on étudie la frontière dans le plan $(\sigma_{xx}, \sigma_{yy})$. La frontière mécanique est définie par

$$\sigma_{VM} = \sigma_y.$$

On peut construire une grille de points et demander au réseau de prédire leur classe.

```

n_grid = 200
SX, SY = np.meshgrid(np.linspace(-10, 150, n_grid), np.linspace(-150, 150, n_grid))
X_grid = torch.tensor(np.column_stack([SX.ravel(), SY.ravel(), np.zeros(n_grid*2)]), dtype=torch.float32)

with torch.no_grad():
    Z = torch.sigmoid(model((X_grid - mean) / std)).numpy().reshape(SX.shape)

plt.figure(figsize=(8, 6))
plt.contourf(SX, SY, Z, levels=20, cmap="coolwarm")
plt.colorbar(label="Probabilite_de_rupture")
plt.xlabel(r"$\sigma_{xx}$ [MPa]")
plt.ylabel(r"$\sigma_{yy}$ [MPa]")
plt.title("Frontiere_de_decision_(Von_Mises)")
plt.show()

```

À vous de réfléchir

Comparer la frontière prédite par le réseau à celle définie par le critère de von Mises.
Pourquoi la frontière apprise n'est-elle pas nécessairement identique à la frontière mécanique exacte ?

4.2 Extension – Critère de Tresca

Dans cette étude on va remplacer von Mises par le critère de Tresca et travailler avec le tenseur de contraintes complet (6 composantes indépendants).

Pour rappel, critère de Tresca est basée sur la contrainte équivalente de Tresca ainsi défini :

$$\sigma_{\text{Tresca}} = \max(|\sigma_1 - \sigma_2|, |\sigma_2 - \sigma_3|, |\sigma_3 - \sigma_1|).$$

, où σ_1, σ_2 et σ_3 sont des contraintes principales.

À vous de jouer

- Modifier le générateur de données pour utiliser le critère de Tresca.
- Réutiliser le même réseau de neurones et comparer les résultats.
- Remplacer la fonction de coût par `nn.BCELoss`. Comment adapter la couche de sortie du réseau ?
- Proposer une extension de cette étude de classification binaire à un problème de classification à trois classes.

5 Travail à rendre

Dans votre compte rendu (rapport ou présentation), veuillez à aborder les points suivants :

1. Description du problème mécanique et présentation des jeux de données générés ainsi comme les scripts utilisés.
2. Description de l'architecture des réseaux de neurones (l'ajout d'autres fonctions d'activation et de méthodes de régularisation est bienvenu).
3. Évolution de la fonction de coût au cours de l'apprentissage et évaluation des performances sur les données d'apprentissage et de test.
4. Étude des capacités d'extrapolation, visualisation et analyse de la frontière de classification apprise.

Les possibilités étant infinies, l'évaluation portera également sur la créativité des solutions proposées, en veillant à établir un lien avec les notions théoriques abordées en CM.